Technical Documentation

Task 5: Source Code Management (SCM) approach and Quality Assurance (QA) strategy for Bayn platform

Source Code Management (SCM) Strategy

The Bayn team uses Git and GitHub for version control with the repository hosted publicly on GitHub. At the beginning of development, the team used multiple repositories, one for each member's given task, for better ease of testing and organization to try out different technologies on the project. When the team decided what technologies to use for each aspect of the project, the development started from scratch on the decided technologies. Now the team uses one Repository with multiple branches.

Branch organization

Branch	Purpose	Rules & Workflow
main	Production-ready code only.	Protected branch. No direct commits. Merged from 'development' branch. Files are pushed here only after teammate/mentor review.
development	Core integration branch for active development.	All feature branches merge here using pull request. Requires teammate review and passing tests.
staging	Pre-production testing and staging environment.	Automatically deployed when code is merged into development. Used for User Acceptance Testing (UAT) and final integration checks before a production release.
feature/name	Isolated development of new features.	**Branch off from:** development **Merge back into:** development

Branch	Purpose	Rules & Workflow
		Naming convention: Use descriptive, hyphenated names (e.g., feature/user-auth, feature/reminder-system).
bugfix/name	Isolation and resolution of identified bugs.	**Branch off from:** development **Merge back into:** development **Naming convention:** Target the issue directly (e.g., bugfix/login-error, bugfix/api-timeout).

Commit standards:

- **Small and focused:** Each commit represents a single logical change.
- **Format:** Commit messages follow a concise prefix format:
    - feature: add calendar
    - fix: correct reminder text
    - docs: update table
- **Long commits:** Used when the update done on the project needs further explanation (more than a line and a half).
    - *Format example:* Short summary line, followed by a blank line, followed by details about the error + error fix.

QA strategy -testing types and tools-

Test Type	Tool	What is Tested	Details
Unit testing	*Ex: PyTest*	Individual backend functions, helper methods, database models, and single frontend UI components.	Ensures that individual, isolated blocks of code operate exactly as expected under standard and

Test Type	Tool	What is Tested	Details
			edge-case inputs before being integrated.
Integration testing	*e.g., PyTest / Postman*	The interaction between different modules (e.g., frontend dashboard components communicating with backend endpoints, database persistence layers).	Verifies that separate modules work smoothly when combined, ensuring that data passes correctly across system boundaries without breaking.
API Testing	*e.g., Postman / Insomnia*	RESTful API endpoints, request/response payloads, HTTP status codes, and authentication/authorization controls.	Validates that the backend securely handles system flows (such as automated digital signatures and scheduler configurations) and returns structured data.
Manual / user testing	*Browser Developer Tools / Notion*	Complete user journeys (e.g., an Idea Owner creating a project, matching with a team, and interacting with the milestone dashboard).	Validates user experience (UX) fluidness, dashboard usability, and overall alignment with functional requirements

Test Type	Tool	What is Tested	Details
			during User Acceptance Testing (UAT).
Regression testing	*e.g., GitHub Actions*	Previously verified features and stable system pathways across the application.	Executed automatically upon pulling new code to guarantee that incoming feature updates or bug fixes do not inadvertently break existing functionality.

Deployment pipeline:

Stage	Trigger	Steps	Purpose
Code push	Developer pushes local commits to a feature/ or bugfix/ branch.	1. Local automated tests run. 2. Code changes are uploaded to the remote GitHub repository.	Isolates individual tasks and ensures the code is safely backed up in the cloud before integration.
Pull Request	Developer opens a PR from their branch into development.	1. Triggers peer review alerts.	Acts as a quality gate to prevent untested, unstable, or unapproved code from

Stage	Trigger	Steps	Purpose
		2. Automated regression tests run. 3. Teammates review line-by-line changes.	entering the shared integration space.
Staging Deploy	Automated trigger when a Pull Request is successfully merged into development.	1. Code is built and compiled. 2. Application deploys automatically to the Staging server. 3. Team conducts final UAT.	Provides an exact replica of the production environment to securely validate real-world platform performance and user journeys.
Production Deploy	Manual approval and merge of the clean development branch into main.	1. Final verification checks. 2. Code is pushed live to the production hosting server. 3. Live post-deployment smoke test.	Safely updates the live system, delivering fully tested, secure, and functional updates to the end users of the Bayn platform.

Environments

- **Development:** Each team member runs the program locally on their machines for active development, prototyping, and initial unit testing.

- **Staging:** A shared cloud environment that mirrors production conditions. It automatically runs the compiled state of the development branch to allow the team to perform collaborative validation, API testing, and User Acceptance Testing (UAT).
- **Production:** The live environment accessible to end users. This environment receives only the code merged into the main branch by an approved pull request with all tests passing and final team/mentor sign-offs complete.